

Exercise 1 - Smoothing and Interpolation

Medical Image Analysis E26

Group G2

This report was completed collaboratively by:

Safiya Aden, André Ribau, Mara Dimitriu, and Sarah Heurtevent

Introduction

In this exercise, we explore smoothing and interpolation using B-spline basis functions. We first implement B-splines of different orders and apply them to a 1D signal extracted from an MRI image. We then investigate how the number and order of the basis functions affect smoothing and interpolation. Finally, the same methods are extended to 2D to smooth and interpolate the MRI image.

Installing & Importing Libraries

Before starting the tasks, the required Python libraries are installed and imported. They will be used for calculations, working with the images, and plotting the results.

```
In [36]: !pip3 install numpy nibabel matplotlib
```

```

Requirement already satisfied: numpy in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (2.4.6)
Requirement already satisfied: nibabel in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (5.4.2)
Requirement already satisfied: matplotlib in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (3.11.2)
Requirement already satisfied: importlib-resources>=5.12 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from nibabel) (7.1.0)
Requirement already satisfied: packaging>=20 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from nibabel) (26.3)
Requirement already satisfied: typing-extensions>=4.6 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from nibabel) (4.16.0)
Requirement already satisfied: contourpy>=1.0.1 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.28.2 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pillow>=9 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (12.3.0)
Requirement already satisfied: pyparsing>=3 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/miniconda3/envs/env_mia/lib/python3.11/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

```

```

In [37]: import numpy as np
import nibabel as nib
from matplotlib import pyplot as plt

np.set_printoptions( suppress=True )
plt.style.use('dark_background') # Dark theme
plt.ion()

```

```
Out[37]: <contextlib.ExitStack at 0x10f496390>
```

We start by reading the 3D MRI scan of the brain and extracting a 2D slice, which will be used throughout the exercise.

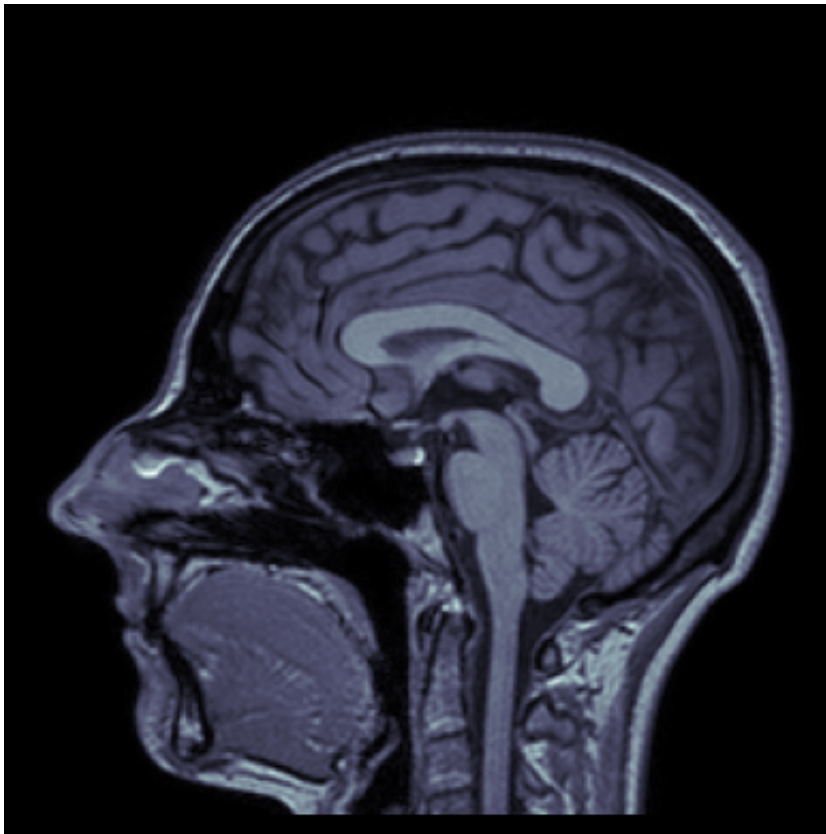
```

In [32]: T1_file= './IXI002-Guys-0828-MPRAGESEN_-s256_-0301-00003-000001-01.
T1_nib = nib.load( T1_file )
data = T1_nib.get_fdata()

img = data[:, :, 75]
T = np.flipud( img.T )

plt.axis('off')
plt.imshow(T, cmap='bone');

```



Next, we extract the middle row of the 2D image and use it as the 1D signal for the first part of the exercise.

```
In [33]: row = round( T.shape[0] / 2 )
         t = T[row, :] #pixel intensity at row y = 128

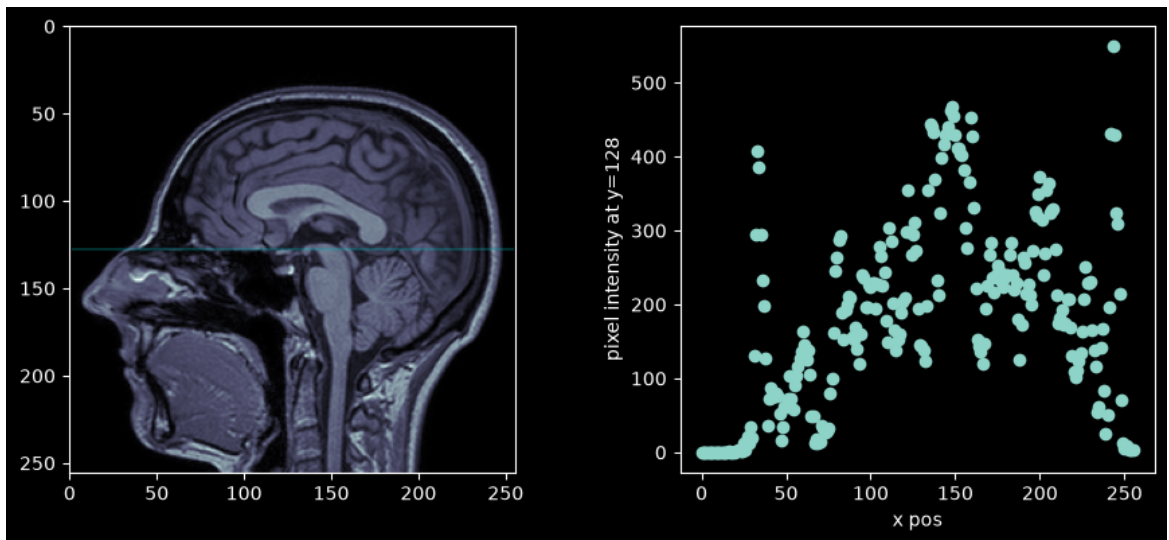
         N = t.shape[0]
         x = np.arange(0, N)

         # plot
         fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10,5))
         fig.tight_layout(pad=5.0)

         ax[0].imshow(T, cmap='bone')
         ax[0].hlines(row, xmin=1, xmax=255, alpha=0.25, color='c')

         ax[1].scatter(x, t)
         ax[1].set_xlabel('x pos')
         ax[1].set_ylabel('pixel intensity at y={}'.format(row))

         plt.show();
```



The left figure shows the selected MRI slice and its middle row at ($y=128$). The right figure shows the pixel intensities along this row as a 1D signal.

Task 1: B-splines basis functions

In this task, we implement a function that evaluates the uniform B-spline of orders 0, 1 and 3 at the locations provided in a vector $\mathbf{x}$. The equations for these B-spline basis functions are given by the following identities, as given in the lecture notes:

$$\beta^0(x) = \begin{cases} 1, & -\frac{1}{2} < x < \frac{1}{2} \\ \frac{1}{2}, & |x| = \frac{1}{2} \\ 0, & \text{otherwise} \end{cases}$$

$$\beta^1(x) = \begin{cases} 1 - |x|, & |x| < 1 \\ 0, & \text{otherwise} \end{cases}$$

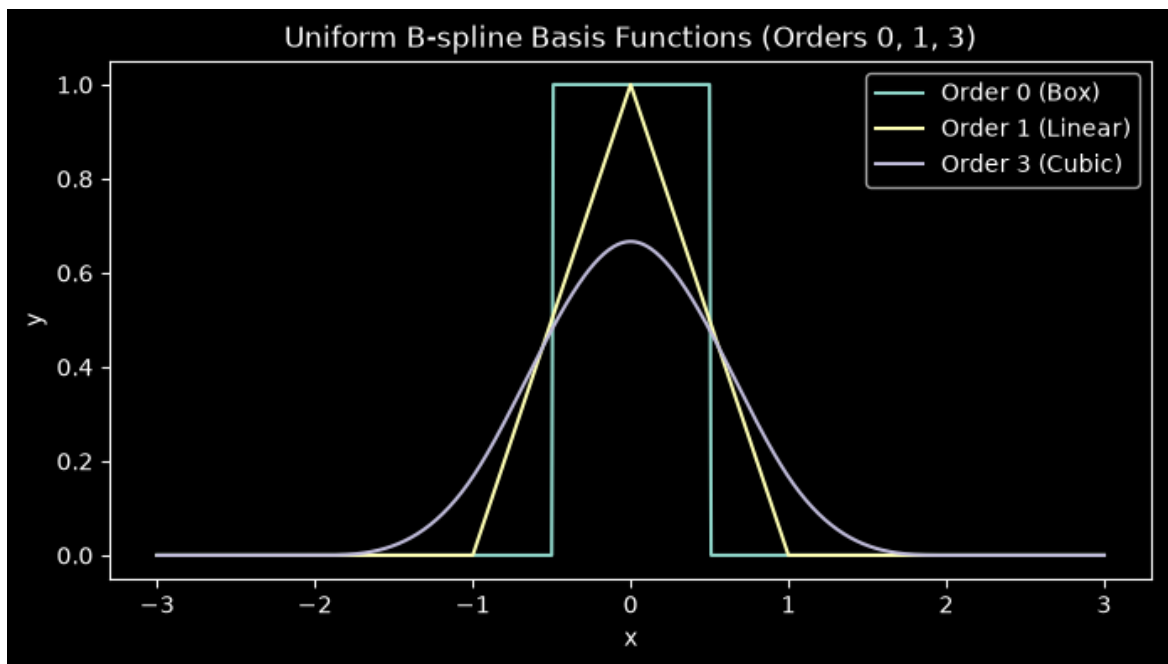
$$\beta^3(x) = \begin{cases} \frac{2}{3} - |x|^2 + \frac{|x|^3}{2}, & |x| < 1 \\ \frac{(2 - |x|)^3}{6}, & 1 \leq |x| < 2 \\ 0, & \text{otherwise} \end{cases}$$

The function returns a vector $\mathbf{y}$ that is the same length as $\mathbf{x}$. We then plot the uniform B-spline of orders 0, 1 and 3 at locations:

$\mathbf{x} = [-3.0, -2.99, -2.98, \dots, 2.99, 3.0]$.

```
In [34]: def eval_BSpline(x, order=0):
          """
          Evaluates the uniform B-spline of order "order" at the location
          Supports orders 0, 1, and 3.
```

```
#####  
  
x = np.asarray(x, dtype=float)  
abs_x = np.abs(x)  
y = np.zeros_like(x)  
  
if order == 0:  
    inside_mask = abs_x < 0.5  
    edge_mask = abs_x == 0.5  
  
    y[inside_mask] = 1.0  
    y[edge_mask] = 0.5  
  
elif order == 1:  
    mask = abs_x < 1.0  
  
    y[mask] = 1.0 - abs_x[mask]  
  
elif order == 3:  
    inner_mask = abs_x < 1.0  
    outer_mask = (abs_x >= 1.0) & (abs_x < 2.0)  
  
    x_inner = abs_x[inner_mask]  
    squared_term = x_inner ** 2  
    cubed_term = x_inner ** 3  
    y[inner_mask] = 2/3 - squared_term + 0.5 * cubed_term  
  
    x_outer = abs_x[outer_mask]  
    base = 2 - x_outer  
    y[outer_mask] = (base ** 3) / 6  
  
return y  
  
x = np.arange(-3.0, 3.0 + 0.005, 0.01)  
  
y0 = eval_BSpline(x, order=0)  
y1 = eval_BSpline(x, order=1)  
y3 = eval_BSpline(x, order=3)  
  
plt.figure(figsize=(8, 4))  
plt.plot(x, y0, label="Order 0 (Box)")  
plt.plot(x, y1, label="Order 1 (Linear)")  
plt.plot(x, y3, label="Order 3 (Cubic)",)  
  
plt.title("Uniform B-spline Basis Functions (Orders 0, 1, 3)")  
plt.xlabel("x")  
plt.ylabel("y")  
plt.legend()  
plt.show()
```



The figure shows the B-spline basis functions for orders 0, 1, and 3. As the order increases, the basis function becomes smoother and extends over a wider range. The order 0 B-spline is a box function, order 1 is piecewise linear, and order 3 gives a smooth cubic curve.

Task 2: Smoothing

Using the function we wrote above for evaluating a uniform B-spline of order 3, we generate $M = 6$ basis functions covering the entire domain $x = 0, 1, \dots, 255$ of the 1D signal $\mathbf{t}$ defined in the introduction (which has length $N = 256$).

The basis functions is be scaled by a factor $h = (N - 1)/(M - 1)$, and shifted to be h units apart. More specifically, we shift by h in order to cover the full range of x from one end to the other, and then we scale by h to "stretch" the basis functions in the appropriate x units. As a results, the basis functions will be given by:

$$\varphi_m(x) = \beta^p \left(\frac{x - mh}{h} \right).$$

```
In [13]: N = 256
          M = 6

          x = np.arange(N)
          h = (N - 1) / (M - 1) # scaling factor

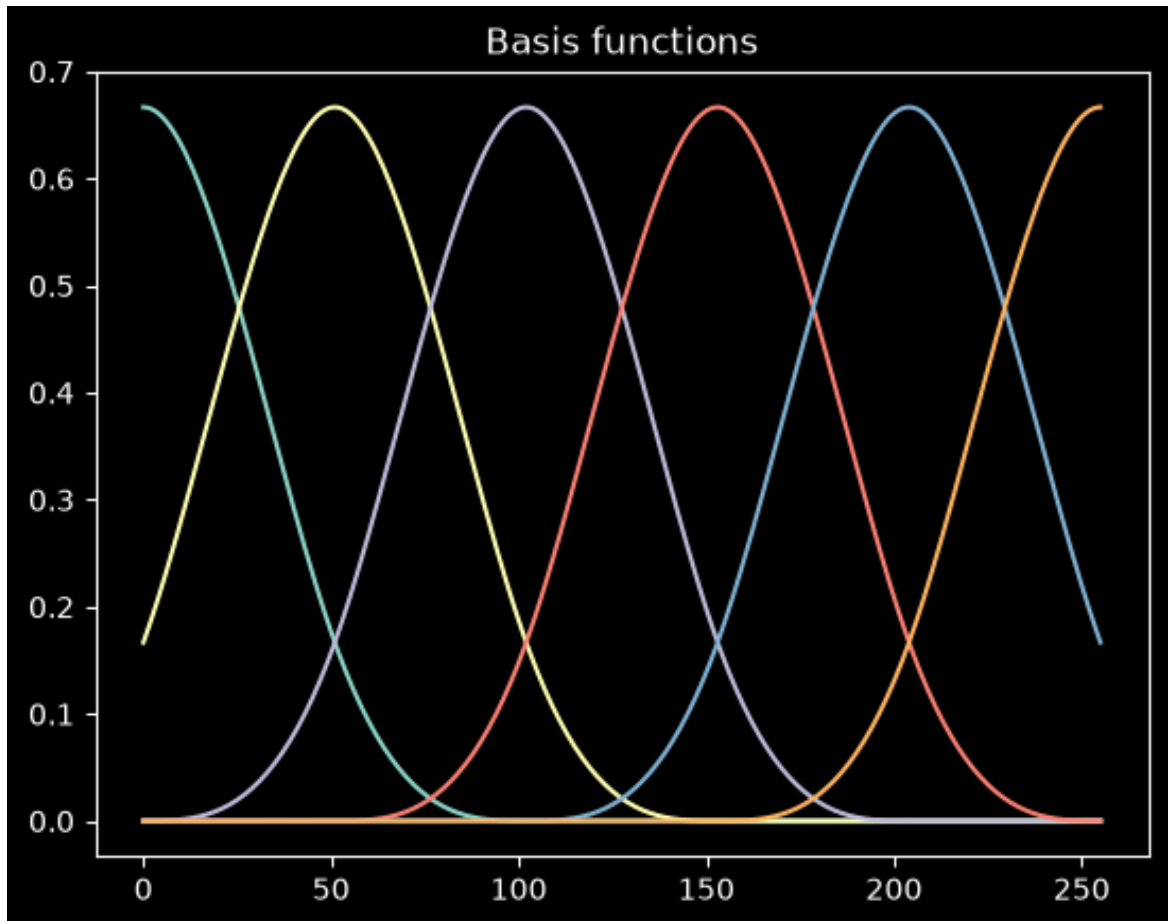
          Phi = np.zeros((N, M))
```

```

for m in range(M):
    center = m * h
    Phi[:, m] = eval_BSpline((x - center) / h, order=3)

plt.plot(x, Phi)
plt.title("Basis functions")
plt.show()

```



The figure shows the six cubic B-spline basis functions distributed across the full signal domain.

We can then calculate the smoothing matrix using the following identity:

$$\mathbf{S} = \mathbf{\Phi}(\mathbf{\Phi}^T \mathbf{\Phi})^{-1} \mathbf{\Phi}^T.$$

In addition, the smoothed signal is given by:

$$\hat{\mathbf{t}} = \mathbf{S} \mathbf{t}.$$

```
In [16]: S = Phi @ np.linalg.inv((np.transpose(Phi)) @ Phi) @ np.transpose(P
```

```
In [17]: t_smooth= S @ t
```

Left figure (below)

In the figure on the left, the smoothing matrix $\mathbf{S}$ is represented visually.

Essentially, the brightness of pixel (i, j) corresponds to the S_{ij} entry of the smoothing matrix, or how much sample j from the original signal contributes to the smoothed value at position i . The bright diagonal band forms because S_{ij} is higher when i and j are closer together, or when the basis functions closer to each other. This illustrates the idea that the smoothed version uses mostly local neighbours.

In addition, the 6 dark "blobs" along the bright diagonal correspond to the $M = 6$ basis functions. This "oscillation" along the edges of the bright diagonal happen because the basis functions overlap their neighbours when spanning the whole domain. Another important feature to point out is the thickness of the diagonal band: because we have a relatively low number of basis functions here, the band becomes wider because, again, the basis functions overlap each other.

Right figure (below)

The figure on the right compares the original signal (blue) to the smoothed out signal (red). Because we have a lower number of basis functions ($M = 6$), the curve in red only captures the general trend of the original signal.

```
In [18]: row = round(T.shape[0] / 2)
         t = T[row, :]

         N = t.shape[0]
         x = np.arange(0, N)

         # plot
         fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 5))
         fig.tight_layout(pad=5.0)

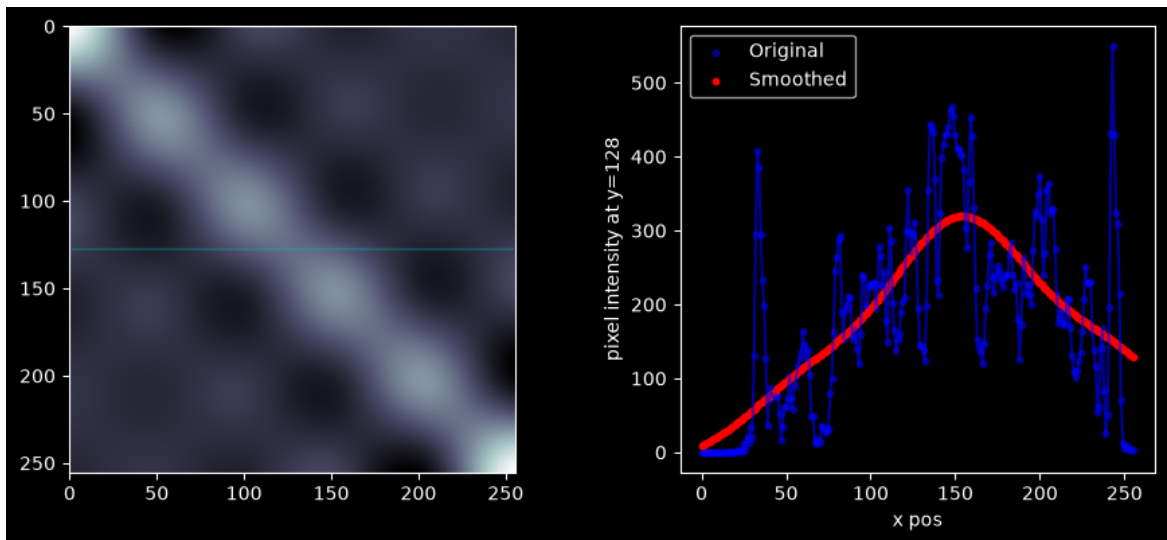
         ax[0].imshow(S, cmap="bone")
         ax[0].hlines(row, xmin=1, xmax=255, alpha=0.25, color="c")

         ax[1].plot(x, t, color="blue", alpha=0.6)
         ax[1].scatter(x, t, label='Original', color='blue', s=10, alpha=0.5)

         ax[1].scatter(x, t_smooth, label="Smoothed", color="red", s=10)

         ax[1].set_xlabel("x pos")
         ax[1].set_ylabel("pixel intensity at y={}".format(row))
         ax[1].legend()

         plt.show()
```



We then repeat the experiments using $M = 16$ and $M = 52$.

```
In [19]: # repeat calculations with M=16
N = 256
M = 16

x = np.arange(N)
h = (N - 1) / (M - 1)

Phi = np.zeros((N, M))

for m in range(M):
    center = m * h
    Phi[:, m] = eval_BSpline((x - center) / h, order=3)

S = Phi @ np.linalg.inv((np.transpose(Phi)) @ Phi) @ np.transpose(P
t_smooth= S @ t

row = round(T.shape[0] / 2)
t = T[row, :]

N = t.shape[0]
x = np.arange(0, N)

# plot
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 5))
fig.tight_layout(pad=5.0)

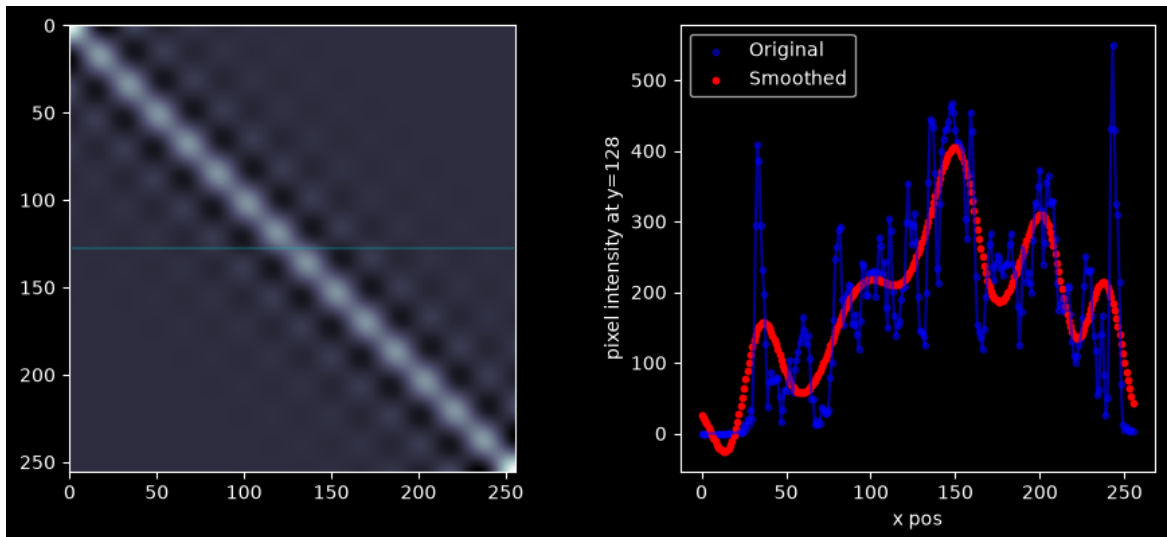
ax[0].imshow(S, cmap="bone")
ax[0].hlines(row, xmin=1, xmax=255, alpha=0.25, color="c")

ax[1].plot(x, t, color="blue", alpha=0.6)
ax[1].scatter(x, t, label='Original', color='blue', s=10, alpha=0.5)

ax[1].scatter(x, t_smooth, label="Smoothed", color="red", s=10)

ax[1].set_xlabel("x pos")
ax[1].set_ylabel("pixel intensity at y={}".format(row))
ax[1].legend()
```

```
plt.show()
```



When increasing the number of basis functions to $M = 16$, we can observe in the figure on the left that the bright band becomes narrower, with more dark "blobs"/oscillations next to it. In this situation, two points have to be much closer together to both land inside the same basis function. In addition, we can also observe a shadow-like feature, parallel to the bright diagonal. This happens because this time the basis functions don't only overlap with their immediate neighbours, but also with more distant ones as well.

In the right figure, we can see that increasing the number of basis functions makes the smoothed out signal to follow the original signal more closely.

```
In [11]: # repeat calculations with M=52
N = 256
M = 52

x = np.arange(N)
h = (N - 1) / (M - 1)

Phi = np.zeros((N, M))

for m in range(M):
    center = m * h
    Phi[:, m] = eval_BSpline((x - center) / h, order=3)

S = Phi @ np.linalg.inv((np.transpose(Phi)) @ Phi) @ np.transpose(P
t_smooth= S @ t

row = round(T.shape[0] / 2)
t = T[row, :]

N = t.shape[0]
x = np.arange(0, N)

# plot
fig, ax = plt.subplots(nrows=1, ncols=2, figsize=(10, 5))
fig.tight_layout(pad=5.0)
```

```

ax[0].imshow(S, cmap="bone")
ax[0].hlines(row, xmin=1, xmax=255, alpha=0.25, color="c")

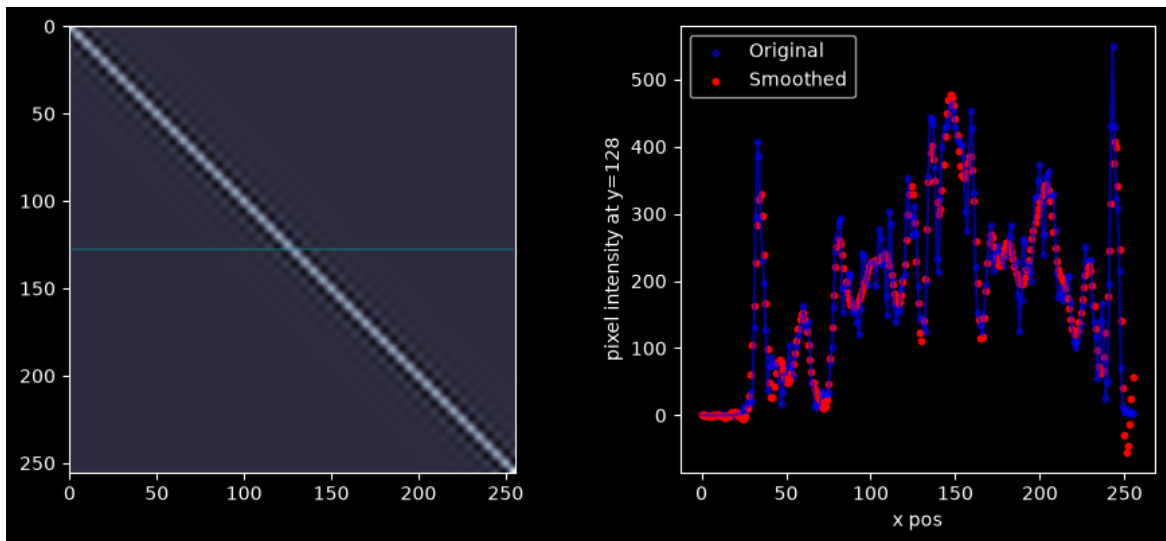
ax[1].plot(x, t, color="blue", alpha=0.6)
ax[1].scatter(x, t, label='Original', color='blue', s=10, alpha=0.5)

ax[1].scatter(x, t_smooth, label="Smoothed", color="red", s=10)

ax[1].set_xlabel("x pos")
ax[1].set_ylabel("pixel intensity at y={}".format(row))
ax[1].legend()

plt.show()

```



In the left figure, when increasing to $M = 52$, the bright diagonal becomes even narrower, as basis functions become even narrower. In addition, the wavy pattern lining the bright diagonal has an even higher frequency. Finally, we can observe in the right figure that the smoothed out signal follows the original signal even more closely, as compared to the previous two situations.

Task 3: Interpolation

We then implement B-spline interpolation of the 1D signal $\mathbf{t}$, by evaluating the function $y(x, \mathbf{w})$ at locations $x = [120, 120.01, 120.02, \dots, 140]$

In the 3 figures below, we can observe the original signal and three different orders of B-splines: 0, 1 and 3. It can be noticed that a higher order gives smoother interpolation curves.

```

In [20]: row = round(T.shape[0] / 2)
         t = T[row, :]
         N = t.shape[0]
         x = np.arange(N)
         xq = np.arange(120, 140, 0.01)

```

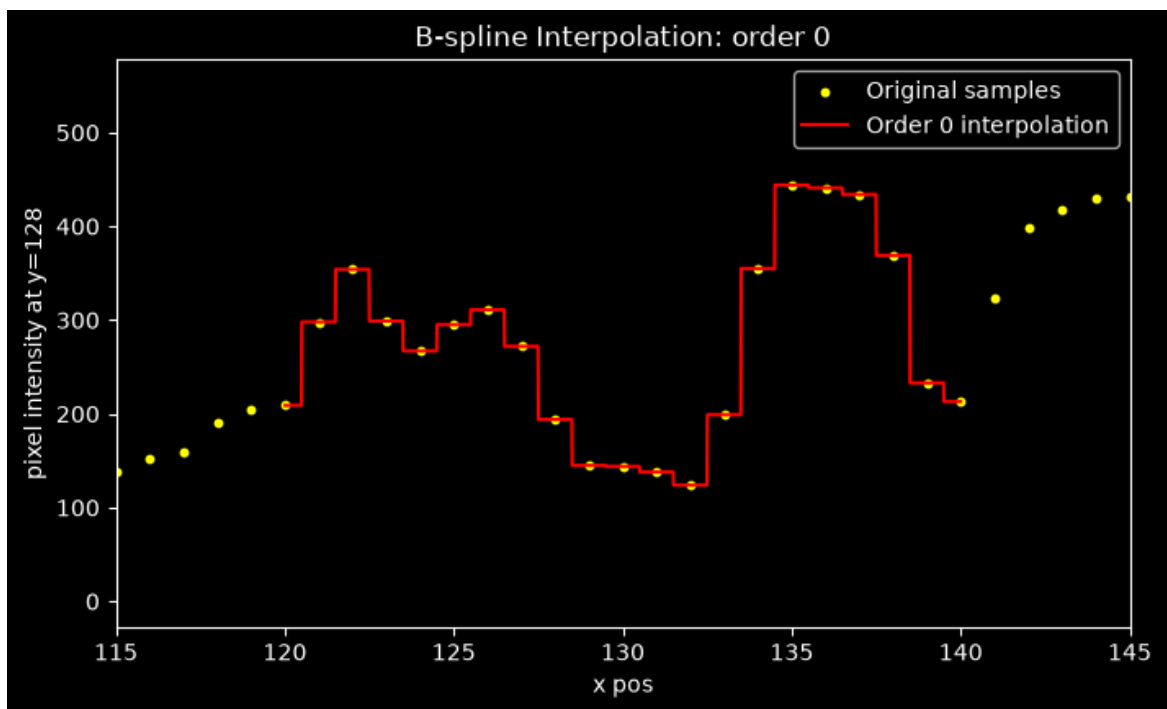
```

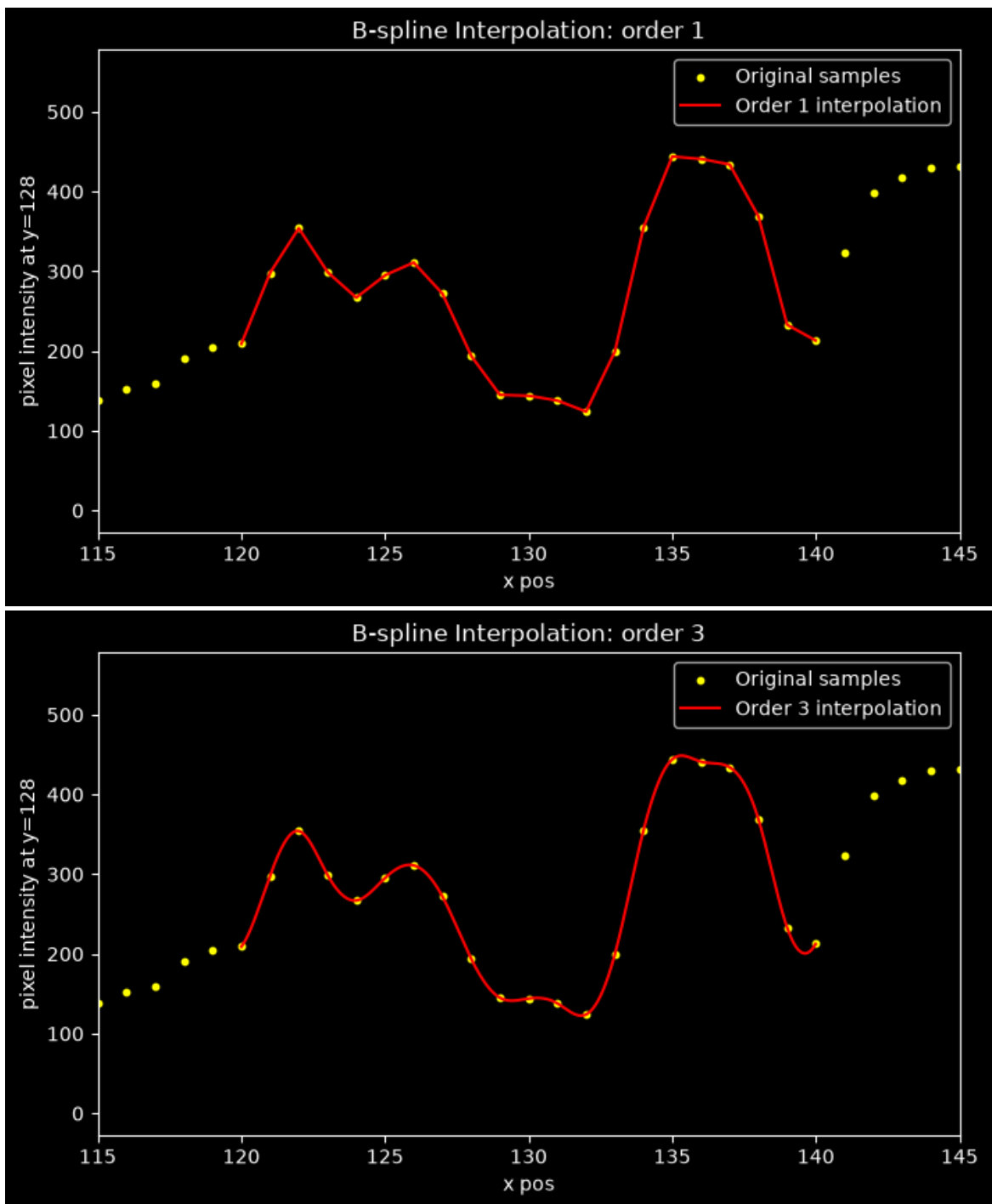
for order in [0, 1, 3]:
    Phi = np.zeros((N, N))
    for m in range(N):
        Phi[:, m] = eval_BSpline(x - m, order=order)
    w = np.linalg.solve(Phi, t)

    Phi_q = np.zeros((len(xq), N))
    for m in range(N):
        Phi_q[:, m] = eval_BSpline(xq - m, order=order)
    y = Phi_q @ w

#plot
plt.figure(figsize=(8, 4.5))
plt.scatter(x, t, label="Original samples", color="yellow", s=1)
plt.plot(xq, y, label=f"Order {order} interpolation", color="red")
plt.xlim(115, 145)
plt.xlabel("x pos")
plt.ylabel(f"pixel intensity at y={row}")
plt.title(f"B-spline Interpolation: order {order}")
plt.legend()
plt.show()

```





Task 4: Smoothing in 2D

Now you are going to smooth the 2D image $\mathbf{T}$, which has dimensions $N_1 \times N_2 = 256 \times 256$, with $M_1 = M_2 = 5$ B-spline-based 1D basis functions of order 3 for each direction.

Use the Kronecker product to produce a $N \times M$ matrix $\Phi = \Phi_2 \otimes \Phi_1$, where:

- $N = N_1 N_2$
- $M = M_1 M_2$.

Show a few 2D basis functions contained in the columns of Φ .

Hints:

- the Kronecker product of two matrices A and B can be calculated with `np.kron(A, B)`
- for converting a vectorized basis function `v` back into its 2D shape, use `v.reshape(N_1, N_2, order='F')`
- the following snippet can be used to show a 2D basis function:

```
plt.figure();
ax = plt.axes(projection='3d');

ax.plot_wireframe(np.arange(N1).reshape(-1, 1),
                  np.arange(N2).reshape(1, -1),
                  basisFunction)
```

```
In [39]: N1 = 256
         N2 = 256
         M1 = 5
         M2 = 5

         x1 = np.arange(N1)
         x2 = np.arange(N2)

         h1 = (N1 - 1) / (M1 - 1)
         h2 = (N2 - 1) / (M2 - 1)

         Phi1 = np.zeros((N1, M1))
         Phi2 = np.zeros((N2, M2))

         for m in range(M1):
             Phi1[:, m] = eval_BSpline((x1 - m*h1) / h1, order=3)

         for m in range(M2):
             Phi2[:, m] = eval_BSpline((x2 - m*h2) / h2, order=3)

         Phi = np.kron(Phi2, Phi1)

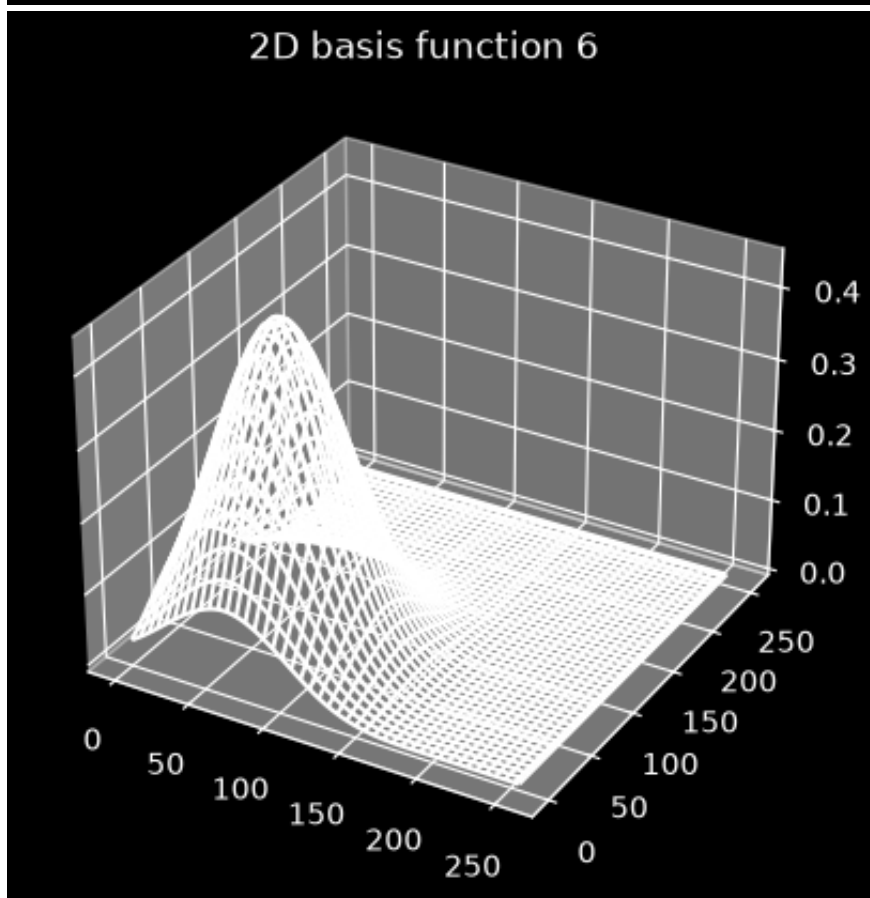
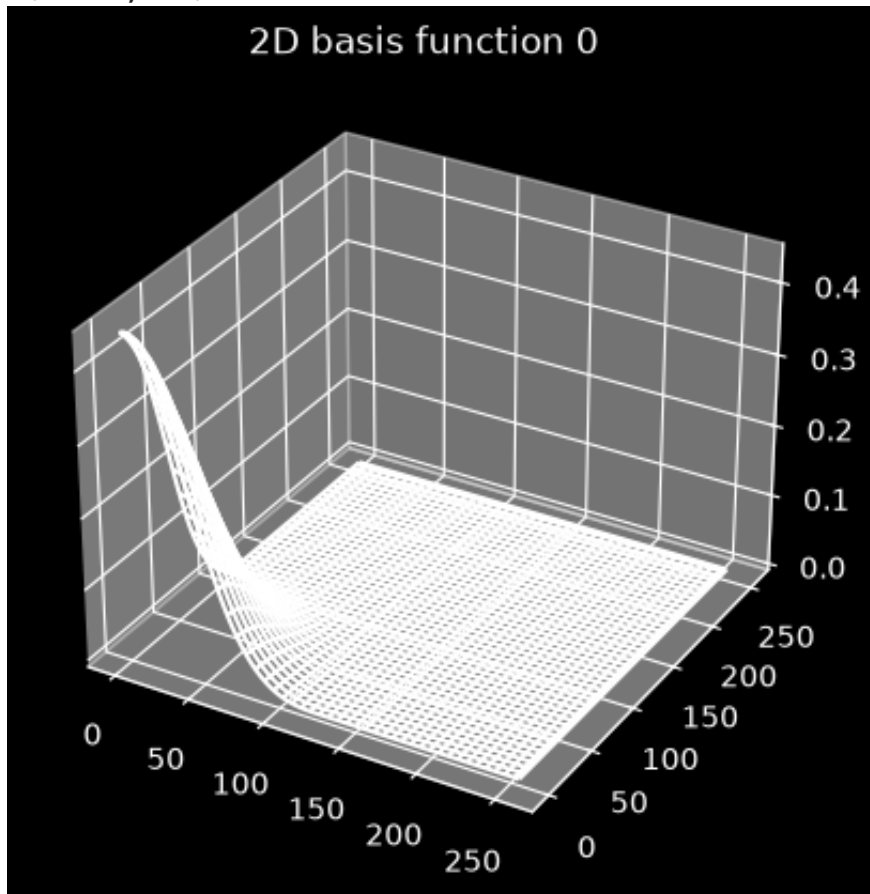
         print(Phi.shape)

         for i in [0, 6, 12, 18, 24]:
             basisFunction = Phi[:, i].reshape(N1, N2, order='F')

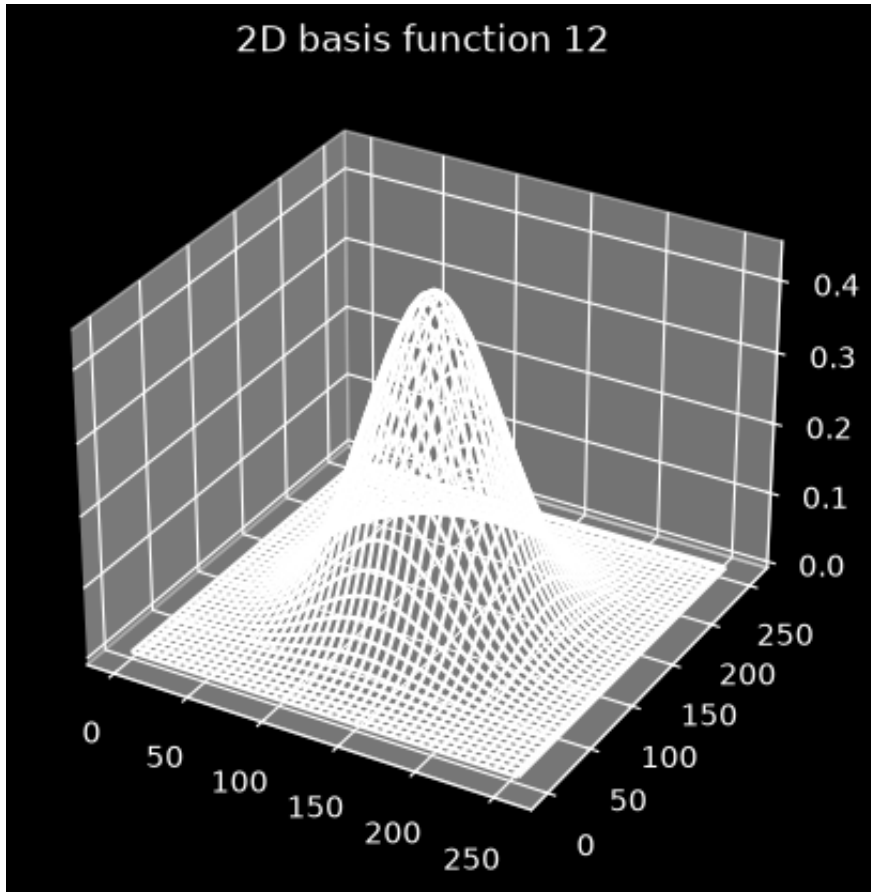
             plt.figure()
             ax = plt.axes(projection='3d')
             ax.plot_wireframe(
                 np.arange(N1).reshape(-1, 1),
                 np.arange(N2).reshape(1, -1),
                 basisFunction
             )
             ax.set_title(f"2D basis function {i}")
```

```
plt.show()
```

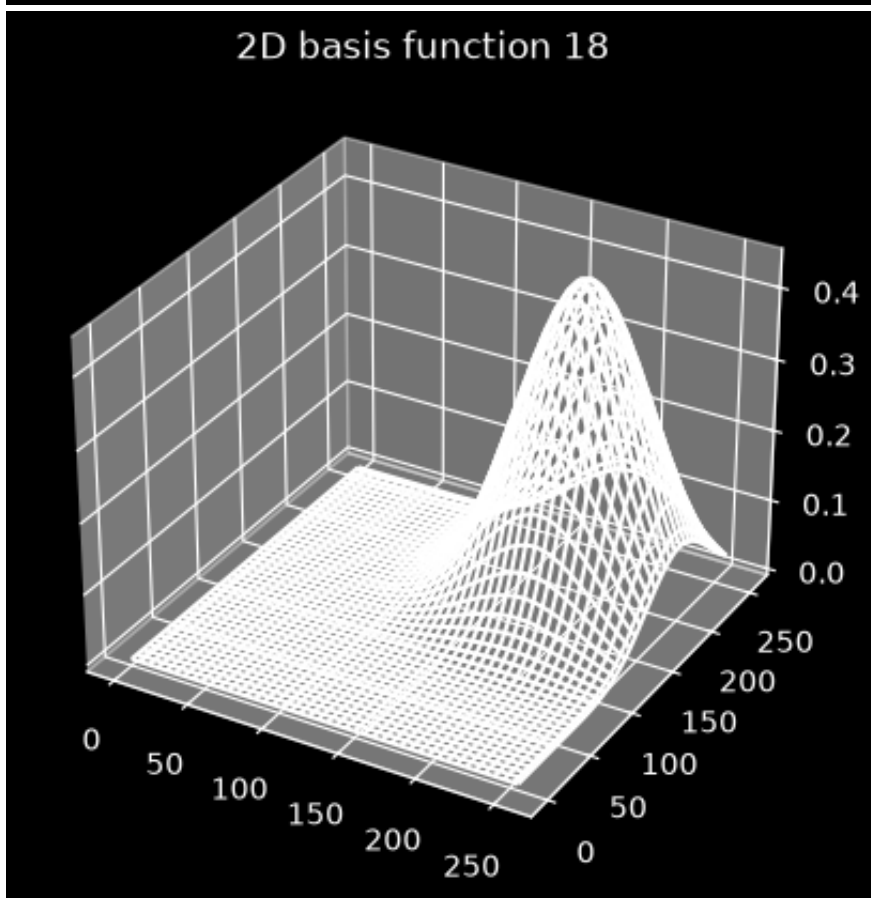
```
(65536, 25)
```

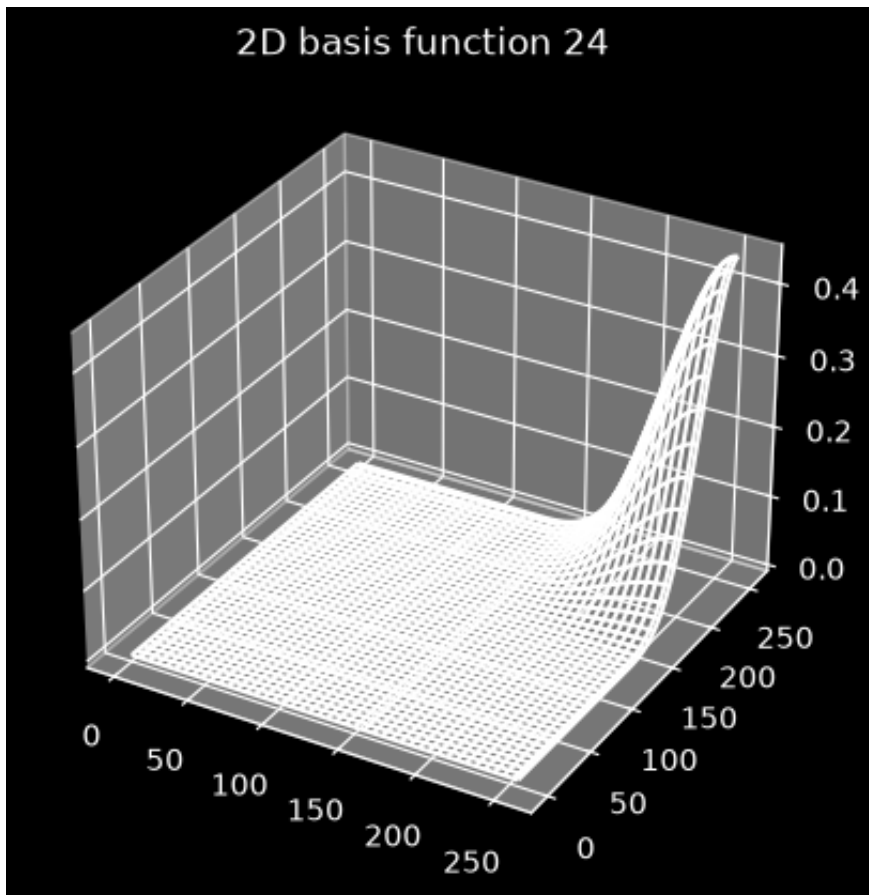


2D basis function 12



2D basis function 18





We create 5 B-spline basis functions in each direction and combine them using the Kronecker product. This gives the 2D basis matrix Φ with dimensions 65536×25 . The figures show examples of the 2D B-spline basis functions contained in the columns of Φ . Each basis function represents the combination of one basis function from each image direction.

Now:

- vectorize the 2D image $\mathbf{T}$ into a 1D signal $\mathbf{t}$
- smooth using Φ
- re-arrange the resulting 1D signal $\hat{\mathbf{t}}$ back into a 2D image $\hat{\mathbf{T}}$
- show the result.

Hints:

- use `T.reshape(-1, 1, order='F')` and `t.reshape(N_1, N_2, order='F')` to vectorize and undo the vectorization.
- do not explicitly compute the smoothing matrix $\mathbf{S}$ as it is very large!! Instead compute the weights $\mathbf{w} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$ and expand those to get $\hat{\mathbf{t}} = \Phi \mathbf{w}$.

```
In [28]: # vectorize the 2D image into a 1D signal
t = T.reshape(-1, 1, order='F')
```

```

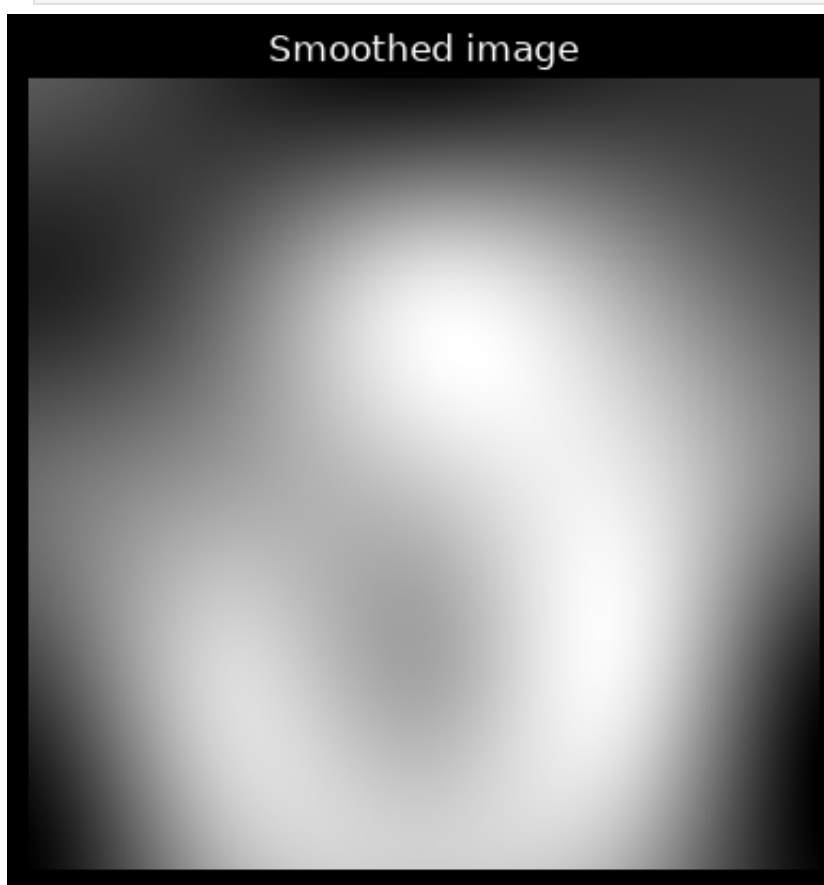
# the weights
w = np.linalg.solve(Phi.T @ Phi, Phi.T @ t)

# the smoothed signal
t_hat = Phi @ w

# make it again into a 2D image
T_hat = t_hat.reshape(N1, N2, order='F')

# plot the result
plt.figure()
plt.imshow(T_hat, cmap='gray')
plt.title('Smoothed image')
plt.axis('off')
plt.show()

```



The original 2D image $\mathbf{T}$ is first vectorized into a 1D signal $\mathbf{t}$. The weights $\mathbf{w}$ are computed using the basis matrix Φ , and the smoothed signal is obtained as $\hat{\mathbf{t}} = \Phi \mathbf{w}$. Finally, the smoothed signal is reshaped back into a 256×256 image $\hat{\mathbf{T}}$ and displayed.

Also perform the same smoothing operation by exploiting the separability of the 2D basis functions, i.e, perform row-wise and then column-wise smoothing instead.

Show the results, and verify that they are identical (use `np.allclose()`)

```

In [29]: # Row-wise smoothing:
w1 = np.linalg.solve(Phi1.T @ Phi1, Phi1.T @ T)

```

```

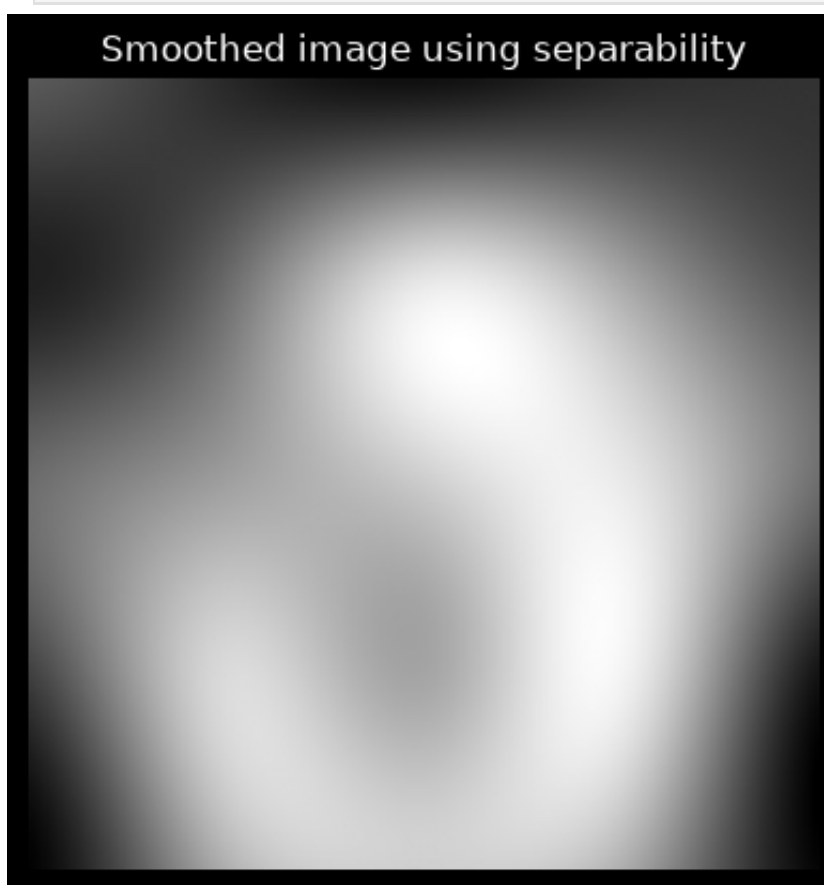
T_row = Phi1 @ w1

# column-wise smoothing:
w2 = np.linalg.solve(Phi2.T @ Phi2, Phi2.T @ T_row.T)
T_separable = (Phi2 @ w2).T

# plot the result
plt.figure()
plt.imshow(T_separable, cmap='gray')
plt.title('Smoothed image using separability')
plt.axis('off')
plt.show()

# Do the two methods give the same result?
print(np.allclose(T_hat, T_separable))

```



True

The image was also smoothed separately in the two directions using Φ_1 and Φ_2 . The resulting image is identical to the result obtained using the full 2D basis matrix Φ . This is verified using `np.allclose()`, which returns `True`.

Task 5: Interpolation in 2D

For B-spline order 0, 1 and 3, compute the weights $\mathbf{W}$ of the 2D interpolation model fitted to $\mathbf{T}$ by exploiting the separability of its basis functions.

Then show, as an image, interpolated function values $y(\mathbf{x}, \mathbf{w})$ for

$\mathbf{x} = (x_1, x_2)^T$ with:

- $x_1 = 120.0, 120.1, 120.2 \dots, 130.0$
- $x_2 = 120.0, 120.1, 120.2 \dots, 130.0$

```
In [ ]: N1, N2 = 256, 256
        M1, M2 = 5, 5

# Builds the design matrix needed for exact interpolation in 1D with
def build_Phi_1D_interp(N, order):

    x = np.arange(N)
    Phi = np.zeros((N, N))
    for m in range(N):
        Phi[:, m] = eval_BSpline(x - m, order=order)
    return Phi

# Fine evaluation grid: 120.0, 120.1, ..., 130.0
x1_eval = np.arange(120, 130.01, 0.1)
x2_eval = np.arange(120, 130.01, 0.1)

orders = [0, 1, 3]
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for ax, order in zip(axes, orders):
    Phi1 = build_Phi_1D_interp(N1, order)
    Phi2 = build_Phi_1D_interp(N2, order)

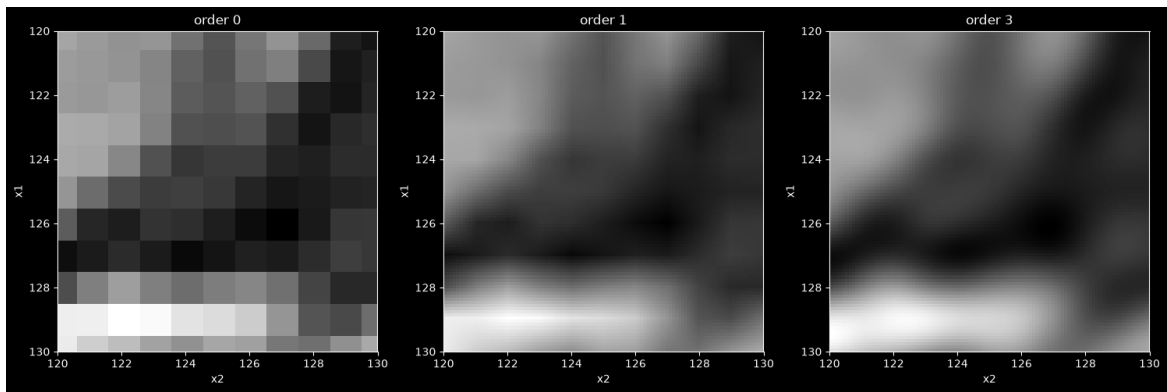
    # Solve for W by exploiting separability: T = Phi1 @ W @ Phi2^T
    A = np.linalg.solve(Phi1, T)
    W = np.linalg.solve(Phi2, A.T).T

    # Basis matrices evaluated at the fine grid (rectangular, not s
    Psi1 = eval_BSpline(x1_eval.reshape(-1, 1) - np.arange(N1).resh
    Psi2 = eval_BSpline(x2_eval.reshape(-1, 1) - np.arange(N2).resh

    Y = Psi1 @ W @ Psi2.T

    ax.imshow(Y, cmap='gray', extent=[120, 130, 130, 120])
    ax.set_title(f"order {order}")
    ax.set_xlabel("x2")
    ax.set_ylabel("x1")

plt.tight_layout()
plt.show()
```



Task 5 — 2D interpolation

We use $M = N$ basis functions per dimension so the model passes exactly through every pixel of $\mathbf{T}$ (interpolation, not smoothing). Since $\Phi = \Phi_2 \otimes \Phi_1$, the condition $\mathbf{T} = \Phi_1 \mathbf{W} \Phi_2^T$ can be solved separably — two 256×256 solves for $\mathbf{W}$ instead of one infeasible 65536×65536 system.

Once $\mathbf{W}$ is known, $y(\mathbf{x}, \mathbf{W}) = \sum_{m_1, m_2} W_{m_1, m_2} \phi_{m_1}(x_1) \phi_{m_2}(x_2)$ defines a continuous surface, evaluable at any (non-integer) location — this is what lets us "zoom in" between pixels. Higher B-spline order gives smoother interpolation: order 0 is blocky (nearest neighbor), order 3 is smooth (cubic).

Conclusion

In this exercise, we used B-spline basis functions for both smoothing and interpolation in 1D and 2D. We observed that using more basis functions allows the smoothed signal to follow the original data more closely, while fewer basis functions result in stronger smoothing. We also saw that higher-order B-splines produce smoother interpolation.

For the 2D image, the Kronecker product was used to construct the 2D basis functions. We also obtained the same smoothing result by applying the operation separately in the two directions, showing the separability of the 2D basis functions. Overall, the exercise demonstrates how B-splines can be used to control the smoothness and level of detail when representing signals and images.